计算机学院_社交媒体与舆情分析_课程实验报告

实验题目：实验 5 来自社交媒体的标签生成		学号：202200130077
日期：2025.11.12	班级：智能 22	姓名：薛润泽
Email：		

实验方法介绍：

复现 Ne-rank_ A novel graph-based keyphrase extraction in twitter。

实验原理：

1、NE-Rank 是一种针对 Twitter 等短文本的无监督图模型关键词抽取算法。它改进了经典的 TextRank 算法，核心思想是在计算图中节点的排序权重时，不仅考虑边的连接关系，还引入了节点自身的重要性。

2、方法上，与 TextRank 类似，首先对文本进行分词、词性标注、去除停用词等预处理，然后将满足特定词性的词作为图的节点。如果两个词在原文的一个固定大小的窗口内共同出现，则在它们对应的节点之间建立一条边。

3、TextRank 算法使用边的权重来计算节点的重要性。公式如下：

$$R(V_i) = (1-d) + d \cdot \sum_{V_j \in In(V_i)} \frac{w_{ji}}{\sum_{V_k \in Out(V_j)} w_{jk}} R(V_j)$$

4、NE-Rank 将节点权重引入到排序公式中，修改后的公式如下：

$$R(V_i) = (1-d) \cdot W(V_i) + d \cdot W(V_i) \cdot \sum_{V_j \in In(V_i)} \frac{w_{ji}}{\sum_{V_k \in Out(V_j)} w_{jk}} R(V_j)$$

5、NE-Rank 利用 Hashtag 提升关键词抽取的准确性。

实验过程描述：（不要求罗列完整源代码）

1、构建一个小型的、高质量的英文推文样本数据集，数据集包含 5 条关于机器学习的模拟推文。

2、文本预处理，将数据集标准化、分词、词性标注、词汇筛选、移除停用词和噪音

3、将词汇间的语义关系模型化为一个图，遍历所有推文，将每个唯一的候选关键词作为一个图节点。使用大小为 3 的滑动窗口，如果两个词出现在窗口内，就在它们对应的节点之间建立一条边。如果两个节点之间已经存在边，则将其权重加 1，边权重最终代表了两个词在整个语料库中共同出现的频次，反映了它们关联的紧密程度。

4、进行 Hashtag 增强。从原始推文中提取所有 Hashtags，实现一个简单的分割逻辑，将其拆分为独立的词。对于从 Hashtag 中分割处的词，我们根据论文公式计算一个“增强分数”。该分数综合考虑了该 hashtag 的全局频率和其对应词在图中的连接度。将计算出的增强分数加到对应词汇的 NE-Rank 得分上。最后将单个词得分转化为输出的关键词短语。

结论分析：

1、算法成功识别出了语料库中的核心概念，如 artificial，intelligence，machine，learning 等。

2、NE-Rank 在文中起到了双重加权机制，使得 artificial intelligence 和 machine learning 得分最高。

3、局限性：短语生成简单，会错过包含非关键词的结构；对于小数据集，TF-IDF 统计意义不大，可能导致节点权重失真。

结论：



代码附录：

```
import re
import string
from collections import defaultdict

import networkx as nx
import nltk
from nltk.corpus import stopwords
from sklearn.feature_extraction.text import TfidfVectorizer
tweets = [
    "Excited to see the new advancements in machine learning and artificial intelligence. #AI #ML",
    "Deep learning models are revolutionizing the field of computer vision. The progress is amazing.",
    "Natural Language Processing is a key part of modern artificial intelligence systems. #NLP",
    "Just attended a great webinar on graph-based algorithms for keyphrase extraction. TextRank is a classic.",
    "The potential of artificial intelligence in healthcare is immense. Machine learning can save lives. #HealthTech"
]
def preprocess(text):
    stop_words = set(stopwords.words('english'))
```

```
    allowed_pos = {'NN', 'NNS', 'NNP', 'NNPS', 'JJ', 'JJS', 'JJR'}
    words = nltk.word_tokenize(text.lower())
    tagged_words = nltk.pos_tag(words)
    filtered_words = []
    for word, tag in tagged_words:
        if tag in allowed_pos and word not in stop_words and word.isalpha():
            filtered_words.append(word)
    return filtered_words
def get_processed_corpus(corpus):
    return [" ".join(preprocess(doc)) for doc in corpus]
def build_graph(processed_docs, window_size=2):
    graph = nx.Graph()
    for doc in processed_docs:
        words = doc.split()
        for i, word in enumerate(words):
            if not graph.has_node(word):
                graph.add_node(word)
            for j in range(i + 1, min(i + window_size, len(words))):
                neighbor_word = words[j]
                if not graph.has_node(neighbor_word):
                    graph.add_node(neighbor_word)
                if graph.has_edge(word, neighbor_word):
                    graph[word][neighbor_word]['weight'] += 1
                else:
                    graph.add_edge(word, neighbor_word, weight=1)
    return graph
def calculate_tf_idf(processed_corpus):
    vectorizer = TfidfVectorizer()
    tfidf_matrix = vectorizer.fit_transform(processed_corpus)
    feature_names = vectorizer.get_feature_names_out()
    word_tfidf = defaultdict(float)
    for doc_idx, doc in enumerate(processed_corpus):
        words = doc.split()
        for word in set(words):
            if word in feature_names:
                word_idx = vectorizer.vocabulary_.get(word)
                word_tfidf[word] += tfidf_matrix[doc_idx, word_idx]
    total_tfidf = sum(word_tfidf.values())
    if total_tfidf == 0:
        return {}
    for word in word_tfidf:
        word_tfidf[word] /= total_tfidf

    return word_tfidf
```

```
def ne_rank(graph, personalization_vector, d=0.85):
    if not personalization_vector:
        return nx.pagerank(graph, alpha=d, weight='weight')
    for node in graph.nodes():
        if node not in personalization_vector:
            personalization_vector[node] = 0.0
    return nx.pagerank(graph, alpha=d, personalization=personalization_vector,
weight='weight')
def extract_hashtags(tweets):
    hashtags = defaultdict(int)
    split_hashtags = []
    total_hashtag_appearances = 0
    for tweet in tweets:
        found = re.findall(r"#(\w+)", tweet)
        for tag in found:
            hashtags[tag] += 1
            total_hashtag_appearances += 1
            split_words = re.findall('[A-Z][^A-Z]*', tag) or [tag]
            split_hashtags.extend([word.lower() for word in split_words])
    return hashtags, split_hashtags, total_hashtag_appearances
def apply_hashtag_boost(word_scores, graph, hashtags,
total_hashtag_appearances):
    boosted_scores = word_scores.copy()
    for tag, count in hashtags.items():
        split_words = re.findall('[A-Z][^A-Z]*', tag) or [tag]
        for word in split_words:
            word = word.lower()
            if word in boosted_scores:
                topical_ratio = count / total_hashtag_appearances
                p_edges = graph.degree(word) / len(graph.edges()) if
graph.has_node(word) and len(graph.edges()) > 0 else 0
                boost = topical_ratio * p_edges
                boosted_scores[word] += boost
    return boosted_scores
def generate_candidate_keyphrases(processed_docs, word_scores, top_n=10):
    top_words = sorted(word_scores.items(), key=lambda x: x[1],
reverse=True)[:top_n]
    top_words_set = set([word for word, score in top_words])
    candidates = set()
    for doc in processed_docs:
        words = doc.split()
        for i in range(len(words)):
            if words[i] in top_words_set:
                phrase = [words[i]]
```

```
                for j in range(i + 1, len(words)):
                    if words[j] in top_words_set:
                        phrase.append(words[j])
                    else:
                        break
                if len(phrase) > 1:
                    candidates.add(" ".join(phrase))
    return list(candidates)
def rank_keyphrases(candidates, word_scores):
    phrase_scores = {}
    for phrase in candidates:
        words = phrase.split()
        score = sum(word_scores.get(word, 0) for word in words) # Using sum for
simplicity, log product is also an option
        phrase_scores[phrase] = score
    return sorted(phrase_scores.items(), key=lambda x: x[1], reverse=True)
if __name__ == "__main__":
    processed_tweets = get_processed_corpus(tweets)
    print("--- Preprocessed Tweets ---")
    print(processed_tweets)
    graph = build_graph(processed_tweets, window_size=3)
    word_tfidf = calculate_tf_idf(processed_tweets)
    word_scores = ne_rank(graph, word_tfidf)
    hashtags, _, total_hashtag_count = extract_hashtags(tweets)
    final_word_scores = apply_hashtag_boost(word_scores, graph, hashtags,
total_hashtag_count)
    print("\n--- Top Ranked Words (after NE-Rank & Boosting) ---")
    sorted_words = sorted(final_word_scores.items(), key=lambda x: x[1],
reverse=True)
    for word, score in sorted_words[:10]:
        print(f"{word}: {score:.4f}")
    candidate_phrases = generate_candidate_keyphrases(processed_tweets,
final_word_scores, top_n=15)
    ranked_phrases = rank_keyphrases(candidate_phrases, final_word_scores)
    print("\n--- Extracted Keyphrases ---")
    for phrase, score in ranked_phrases[:5]:
        print(f"{phrase}: {score:.4f}")
```